Johannes Gutenberg-Universität Mainz

Dezernat Hochschulentwicklung (HE)

KI-Assistent Web-Anwendung für die Hochschulverwaltung

Konzeptpapier

Machbarkeitsstudie und Funktionsbeschreibung

Autor: Paul Baselt
Kontakt: baselt@uni-mainz.de
Institution: Johannes Gutenberg-Universität Mainz
Status: Entwurf / Machbarkeitsnachweis
Version: 1.0
Datum: 9. April 2026

Dieses Dokument ist für den internen Gebrauch an der JGU Mainz bestimmt.

Inhaltsverzeichnis

1 Zusammenfassung **2**

2 Motivation und Problemstellung **2**

3 Wissenschaftlicher Hintergrund **2**
3.1 Prompt Engineering: Überblick und Taxonomien 2
3.2 Diversität und Qualität von LLM-Ausgaben 3
3.3 Mensch-KI-Kollaboration und Synergieeffekte 3

4 Zielsetzung **3**

5 Funktionsumfang **4**
5.1 Vorgefertigte KI-Agenten 4
5.2 Datei-Support 4
5.3 Weitere Funktionen 4

6 Konzept: Prompt-Bibliothek **5**
6.1 Motivation 5
6.2 Zweistufiges Bibliotheksmodell 5
6.3 Prompt-Template-Struktur 5
6.4 Nutzer-Workflow 6
6.5 Technische Umsetzungsoptionen 6

7 Technische Architektur **6**
7.1 Systemkomponenten 6
7.2 Voraussetzungen 7

8 Datenschutz und Sicherheit **7**

9 Roadmap und offene Punkte **7**
9.1 Abgeschlossen 7
9.2 Geplante Erweiterungen 7

10 Fazit **8**

Zusammenfassung

Dieses Konzeptpapier beschreibt die Entwicklung und den Einsatz einer KI-basierten Web-Anwendung (»Verwaltungsagent«) für die Johannes Gutenberg-Universität Mainz. Die Anwendung ermöglicht Mitarbeiterinnen und Mitarbeitern einen niedrigschwelligen Zugang zu den KI-Services der Universität über eine benutzerfreundliche Oberfläche. Vorgefertigte Aufgabenvorlagen (Agenten) reduzieren den Aufwand für die Formulierung von Prompts und erhöhen die Qualität der KI-Antworten für typische Verwaltungsaufgaben. Die vorliegende Version ist als Machbarkeitsnachweis konzipiert und bildet die Grundlage für eine mögliche institutionelle Weiterentwicklung.

Motivation und Problemstellung

Hochschulen stehen zunehmend vor der Herausforderung, KI-Werkzeuge sinnvoll und sicher in bestehende Arbeitsabläufe zu integrieren. Obwohl die JGU Mainz über eine eigene KI-Infrastruktur verfügt (ki-chat.uni-mainz.de), erfordert deren direkte API-Nutzung technische Kenntnisse, die im Verwaltungsbereich häufig nicht vorhanden sind.

Konkrete Problemfelder:

- Fehlende Benutzeroberfläche für die direkte API-Nutzung ohne technisches Vorwissen
- Inkonsistente Prompt-Qualität führt zu variablen Ergebnissen
- Kein institutioneller Standard für typische Verwaltungsaufgaben (Zusammenfassung, Übersetzung, E-Mail-Erstellung)
- Datenschutzbedenken bei der Nutzung externer KI-Dienste (z. B. ChatGPT)

Wissenschaftlicher Hintergrund

Die Gestaltung des Verwaltungsagenten basiert auf aktuellen Erkenntnissen der Forschung zu Prompt Engineering, In-Context Learning und Mensch-KI-Zusammenarbeit. Im Folgenden werden die zentralen Referenzarbeiten kurz vorgestellt.

Prompt Engineering: Überblick und Taxonomien

Sahoo u. a. (2024) liefern einen systematischen Überblick über mehr als 41 Prompt-Engineering-Techniken, kategorisiert nach Anwendungsbereichen. Neben grundlegenden Ansätzen wie Zero-Shot- und Few-Shot-Prompting werden fortgeschrittene Methoden wie Chain-of-Thought (CoT), Tree-of-Thoughts und Retrieval-Augmented Generation (RAG) behandelt. Für jede Technik werden Stärken, Schwächen sowie eingesetzte Modelle und Datensätze dokumentiert. Die Arbeit verdeutlicht, dass die Qualität von Prompts maßgeblich die Ausgabequalität von LLMs bestimmt – ein zentrales Argument für den Einsatz vorgefertigter, erprobter Prompt-Vorlagen im Verwaltungsagenten.

Das bisher umfangreichste Survey zu diesem Thema stammt von Schulhoff u. a. (2025): Der *Prompt Report* definiert eine einheitliche Terminologie mit 33 Fachbegriffen, identifiziert 58 textbasierte LLM-Techniken sowie 40 weitere für andere Modalitäten und

diskutiert Sicherheitsrisiken (Prompt Hacking) und Alignment-Probleme. Die Arbeit zeigt, dass eine strukturierte Herangehensweise an Prompts – wie sie der Verwaltungsagent durch vordefinierte Agenten umsetzt – die Nutzbarkeit und Zuverlässigkeit von KI-Systemen erheblich verbessert.

Hooper (2025) ergänzen diesen Überblick mit einem Fokus auf In-Context Learning (ICL): Modelle können neue Aufgaben allein durch Beispiele im Prompt erlernen, ohne Modellparameter anzupassen. Effektives Prompt Engineering kann die Modellleistung auf Benchmarks um 20–40 % steigern. Die Arbeit kategorisiert Techniken in manuelle (Templates, Instruktionen), automatisierte (Soft Prompts, RL-basierte Optimierung) und hybride Ansätze und identifiziert Prompt-Sensitivität als zentrale Herausforderung für den Praxiseinsatz.

Diversität und Qualität von LLM-Ausgaben

Zhang u. a. (2025) zeigen, dass Post-Training-Alignment-Verfahren (RLHF) die Ausgabenvielfalt von LLMs durch *Mode Collapse* reduzieren. Als Ursache identifizieren sie einen *Typizitätsbias* in Präferenzdaten: Annotatorinnen und Annotatoren bevorzugen systematisch vertraute, typische Antworten, was das Modell auf wenige häufige Ausgaben einengt. Als trainingsfreie Lösung schlagen die Autoren *Verbalized Sampling* (VS) vor: Das Modell wird aufgefordert, explizit eine Verteilung von Antworten mit Wahrscheinlichkeiten zu generieren statt einer einzelnen Antwort. VS steigert die Ausgabenvielfalt in kreativen Aufgaben um den Faktor 1,6–2,1, ohne Faktentreue oder Sicherheit zu beeinträchtigen. Diese Erkenntnis ist für den Verwaltungsagenten relevant, wenn es darum geht, unterschiedliche Formulierungsvarianten (z. B. für E-Mails) bereitzustellen.

Mensch-KI-Kollaboration und Synergieeffekte

Anonymous (2025) untersuchen mittels eines Bayesianischen Item-Response-Theory-Frameworks, wie stark verschiedene KI-Modelle die menschliche Problemlöseleistung verbessern. Die Studie zeigt: GPT-4o steigert die Leistung von Nutzenden um durchschnittlich 29 Prozentpunkte, Llama-3.1-8B um 23 Prozentpunkte – gemessen an einem Datensatz mit 667 Teilnehmenden. Entscheidend ist dabei die individuelle *Theory of Mind*: Nutzende, die besser in der Lage sind, die Perspektive anderer einzunehmen und zu antizipieren, erzielen eine überlegene kollaborative Leistung mit KI. Für schwächere Nutzende wirkt KI als Leistungsausgleich (»Leveling the Playing Field«). Diese Befunde unterstreichen den Mehrwert eines niedrigschwelligen KI-Zugangs, wie ihn der Verwaltungsagent bietet.

Zielsetzung

Die Anwendung verfolgt folgende Ziele:

1. **Zugänglichkeit:** Bereitstellung einer intuitiven Web-Oberfläche für die KI-API der JGU, ohne Programmierkenntnisse vorauszusetzen.
2. **Standardisierung:** Vorgefertigte, erprobte Prompts für häufige Aufgaben sichern eine konsistente Ausgabequalität.

3. **Datenschutz:** Ausschließliche Nutzung der universitätseigenen KI-Infrastruktur; keine Übertragung von Daten an externe Dienste.
4. **Erweiterbarkeit:** Modularer Aufbau ermöglicht die spätere Integration weiterer Agenten und Funktionen.

Funktionsumfang

Vorgefertigte KI-Agenten

Die Anwendung stellt sechs spezialisierte Agenten bereit, die für häufige Verwaltungsaufgaben optimiert sind:

Agent	Beschreibung
Text zusammenfassen	Erstellt strukturierte Zusammenfassungen aus langen Dokumenten
Text übersetzen	Professionelle Übersetzungen in 6 Sprachen (DE, EN, FR, ES, IT, NL)
E-Mail schreiben	Generiert formelle E-Mails aus Stichpunkten
Daten analysieren	Strukturierte Analyse mit Handlungsempfehlungen
Thema erforschen	Umfassende Recherche zu beliebigen Themen
Code erstellen	Sauberer, kommentierter Code in 7 Programmiersprachen

Tabelle 1: Übersicht der implementierten KI-Agenten

Datei-Support

- **PDF-Dateien** (.pdf): Automatische Textextraktion via PDF.js
- **Textdateien** (.txt): Direkter Import
- **Multi-Upload**: Bis zu 3 Dateien gleichzeitig

Hinweis: Große Dokumente werden derzeit auf 12.000 Zeichen (ca. 3.000 Tokens) begrenzt. DOCX-Dateien werden noch nicht unterstützt.

Weitere Funktionen

- Dynamische Modell-Auswahl direkt von der API
- Sichere API-Key-Verwaltung (lokaler Browser-Speicher)
- Verlaufshistorie der letzten 50 Ergebnisse
- Export-Funktionen (Kopieren, Speichern)
- Debug-Konsole für technische Diagnose
- Responsives Design (Desktop und Mobilgeräte)

Konzept: Prompt-Bibliothek

Motivation

Die aktuelle Implementierung bietet sechs fest kodierte Agenten mit unveränderlichen Prompts. Für den Praxiseinsatz in der Hochschulverwaltung ist jedoch eine größere Flexibilität notwendig: Mitarbeitende in verschiedenen Dezernaten haben unterschiedliche Anforderungen, bevorzugte Formulierungen und wiederkehrende Aufgaben, die sich nicht durch ein einheitliches Prompt-Set abdecken lassen. Gleichzeitig belegen Sahoo u. a. (2024) und Schulhoff u. a. (2025), dass gut strukturierte, aufgabenspezifische Prompts die Qualität von KI-Ausgaben erheblich verbessern. Die Erweiterung um eine persönliche und institutionelle Prompt-Bibliothek ist daher ein zentraler nächster Entwicklungsschritt.

Zweistufiges Bibliotheksmodell

Das Konzept sieht zwei Ebenen von Prompt-Sammlungen vor, die sich ergänzen:

Institutionelle Bibliothek (schreibgeschützt)
: Zentral verwaltete, erprobte Prompts für hochschulweite Standardaufgaben. Diese werden durch Administratoren gepflegt und stehen allen Nutzenden zur Verfügung. Nutzende können diese Prompts als Vorlage in ihre persönliche Bibliothek übernehmen und dort anpassen.

Persönliche Bibliothek (vollständig bearbeitbar)
: Individuelle Prompt-Sammlung je Nutzerin bzw. Nutzer. Eigene Prompts können neu erstellt, aus der institutionellen Bibliothek kopiert, bearbeitet, kategorisiert, exportiert und gelöscht werden.

Prompt-Template-Struktur

Prompts werden als Templates mit variablen Platzhaltern gespeichert. Vor dem Absenden werden die Platzhalter durch ein dynamisches Formular befüllt. Ein Prompt-Datensatz enthält folgende Felder:

Feld	Typ	Beschreibung
`id`	String (UUID)	Eindeutige Kennung
`name`	String	Anzeigename des Prompts
`kategorie`	String	Thematische Gruppe (z. B. Kommunikation, Analyse)
`template`	String	Prompt-Text mit Platzhaltern (`{{Variable}}`)
`variablen`	Array	Liste der enthaltenen Platzhalter-Namen
`quelle`	Enum	`institutionell` oder `persönlich`
`erstellt_am`	Datum	Erstellungsdatum

Tabelle 2: Datenstruktur eines Prompt-Templates

Beispiel: Ein Template für mehrsprachige E-Mails könnte lauten:

> *»Schreibe eine `{{Tonalität}}` E-Mail auf `{{Sprache}}` über folgenden Sachverhalt: `{{Inhalt}}`«*

Beim Aufruf dieses Prompts erscheint ein Formular mit drei Feldern (*Tonalität*, *Sprache*, *Inhalt*), das der Nutzer ausfüllt, bevor der fertige Prompt an das Modell gesendet wird.

Nutzer-Workflow

Der Ablauf aus Nutzerperspektive gliedert sich in vier Schritte:

1. **Auswählen:** Nutzende wählen einen Prompt aus der institutionellen oder persönlichen Bibliothek – gefiltert nach Kategorie oder per Freitextsuche.
2. **Anpassen (optional):** Platzhalter werden in einem dynamisch generierten Formular befüllt. Der resultierende Prompt ist vor dem Absenden noch manuell editierbar.
3. **Absenden:** Der fertige Prompt wird wie bisher an das gewählte Modell über die JGU-API gesendet.
4. **Speichern (optional):** Nutzende können den angepassten Prompt unter einem eigenen Namen in ihrer persönlichen Bibliothek speichern.

Technische Umsetzungsoptionen

Die Speicherung der persönlichen Bibliothek kann in drei Ausbaustufen erfolgen:

Stufe	Technologie	Eigenschaften
1	`localStorage` / `IndexedDB`	Kein Server erforderlich; Daten nur im Browser; geeignet als erster Prototyp
2	Lokale JSON-Datei (Export/Import)	Portabilität zwischen Geräten; manueller Austausch möglich
3	Server-seitige Datenbank (z. B. SQLite, PostgreSQL)	Geräteübergreifend; Teilen von Prompts; Rechteverwaltung; Integration in JGU-Infrastruktur

Tabelle 3: Speicheroptionen für die persönliche Prompt-Bibliothek

Für den Machbarkeitsnachweis wird **Stufe 1** (IndexedDB im Browser) empfohlen, da sie ohne Serverinfrastruktur auskommt und dennoch größere Datenmengen als `localStorage` verwalten kann. Eine Migration zu Stufe 3 ist zu einem späteren Zeitpunkt möglich, ohne die Kernlogik zu verändern.

Technische Architektur

Systemkomponenten

Die Anwendung besteht aus drei Hauptkomponenten:

1. **Frontend** (`JGU_Agenten.html`, `app.js`, `styles.css`): Browserbasierte Benutzeroberfläche ohne externe Framework-Abhängigkeiten.
2. **CORS-Proxy** (`proxy-server.js`): Lokaler Node.js-Server, der Browser-Sicherheitsrestriktionen (CORS) umgeht und API-Anfragen weiterleitet.
3. **KI-API der JGU**: ki-chat.uni-mainz.de/api – die universitätseigene KI-Infrastruktur (OpenAI-kompatibel).

Voraussetzungen

- Node.js (v14 oder höher) für den Proxy-Server
- Moderner Webbrowser (Chrome, Firefox, Edge)
- API-Key von ki-chat.uni-mainz.de

Datenschutz und Sicherheit

- Alle API-Anfragen werden ausschließlich an die universitätseigene Infrastruktur gesendet.
- Der API-Key wird ausschließlich im lokalen Browser-Speicher (LocalStorage) gespeichert und nicht an Dritte übertragen.
- Die Anwendung nutzt keinen externen CDN und lädt keine Ressourcen von Drittanbietern nach (außer PDF.js).
- Die Einhaltung der JGU-Datenschutzrichtlinien ist bei einem möglichen Produktiveinsatz zu prüfen.

Roadmap und offene Punkte

Abgeschlossen

- ✓ Dynamische Modell-Auswahl
- ✓ Debug-Logging-System
- ✓ CORS-Proxy-Lösung
- ✓ Start-Skripte für Windows und Linux/macOS
- ✓ PDF-Textextraktion via PDF.js

Geplante Erweiterungen

- **Prompt-Bibliothek** (persönlich + institutionell) mit Template-System und Platzhaltern (siehe Abschnitt 5)
- Download der Antworten als Word- oder PDF-Dokument
- DOCX-Dateiverarbeitung
- Chunking für große Dokumente (statt Kürzung)

- Streaming-Support für schnellere Antwortanzeige
- Dark Mode

Fazit

Die entwickelte Web-Anwendung demonstriert, dass eine benutzerfreundliche Oberfläche für die KI-Services der JGU mit geringem technischem Aufwand realisierbar ist. Der Machbarkeitsnachweis zeigt das Potenzial für eine breitere institutionelle Nutzung. Eine Weiterentwicklung in Richtung produktiver Einsatz erfordert die Einbindung der zuständigen Stellen (ZDV, Datenschutzbeauftragter) sowie eine systematische Bedarfserhebung bei den Zielgruppen in der Verwaltung.

Literatur

Anonymous (2025). „Quantifying Human-AI Synergy“. In: *Under double-blind review*. Preprint, eingereicht zur Begutachtung.

Hooper, Aaron (2025). „Prompt Engineering and In-Context Learning: A Comprehensive Survey of Techniques, Applications, and Future Directions“. In: *Preprint*. Unveröffentlichtes Manuskript.

Sahoo, Pranab u. a. (2024). „A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications“. In: *arXiv preprint arXiv:2402.07927*. Version 2, März 2025.

Schulhoff, Sander u. a. (2025). „The Prompt Report: A Systematic Survey of Prompt Engineering Techniques“. In: *arXiv preprint arXiv:2406.06608*. Version 6, Februar 2025.

Zhang, Jiayi u. a. (2025). „Verbalized Sampling: How to Mitigate Mode Collapse and Unlock LLM Diversity“. In: *arXiv preprint arXiv:2510.01171*. Version 3, Oktober 2025.